

电子设计基础训练

Arduino接口与显示

北京航空航天大学
电子信息工程学院



Arduino接口与显示

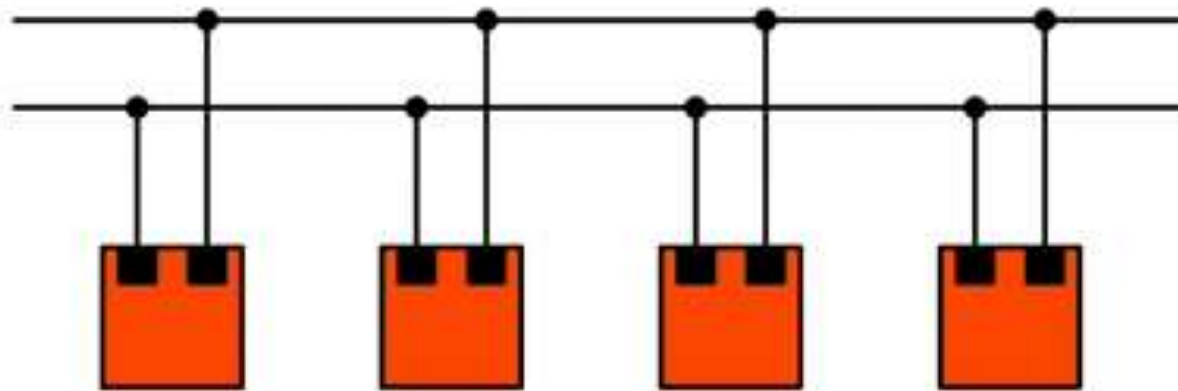
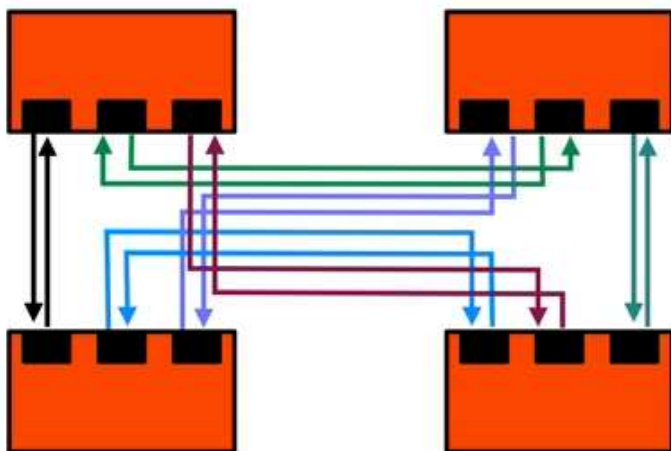
- Arduino的I2C通信
- OLED显示屏
- Arduino库文件
- 字符与图像显示
- 实验：Arduino小游戏



Arduino的I2C通信

■ I2C简介

- I2C (Inter-Integrated Circuit) 是I2CBus简称，中文叫集成电路间总线，它是一种串行通信总线。
- I2C主机可以与多从机通讯，多设备不增加连线数
- UART串口端对端传输，多设备时连接复制



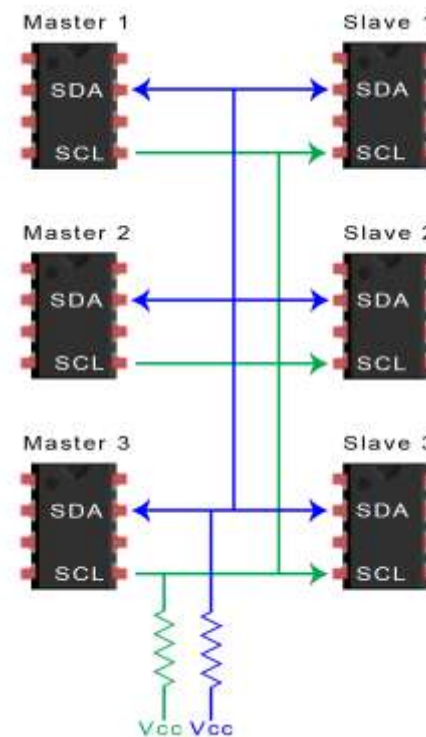
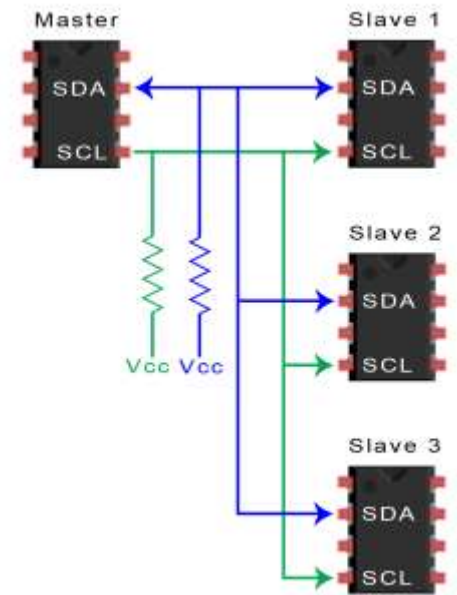
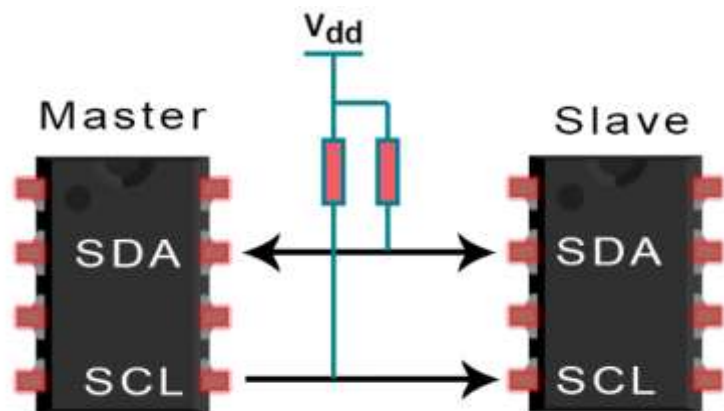


Arduino的I2C通信

■ I2C简介

➤ I2C采用两线制总线协议标准

- 一根时钟信号线SCL (Serial Clock Line)
- 一根双向数据线SDA (Serial Data Line)
- 支持一主多从或者多主连接
- 采用地址区分不同设备
- 半双工：双向的数据交换都在一根数据线上完成





Arduino的I2C通信

■ I2C通信的数据帧结构示意图



设备地址：区分各个从设备的地址码

应答信号：从设备发送给主机信号，表示接收完成

起始位：开始进行一次传输的标志

停止位：结束一次传输的标志

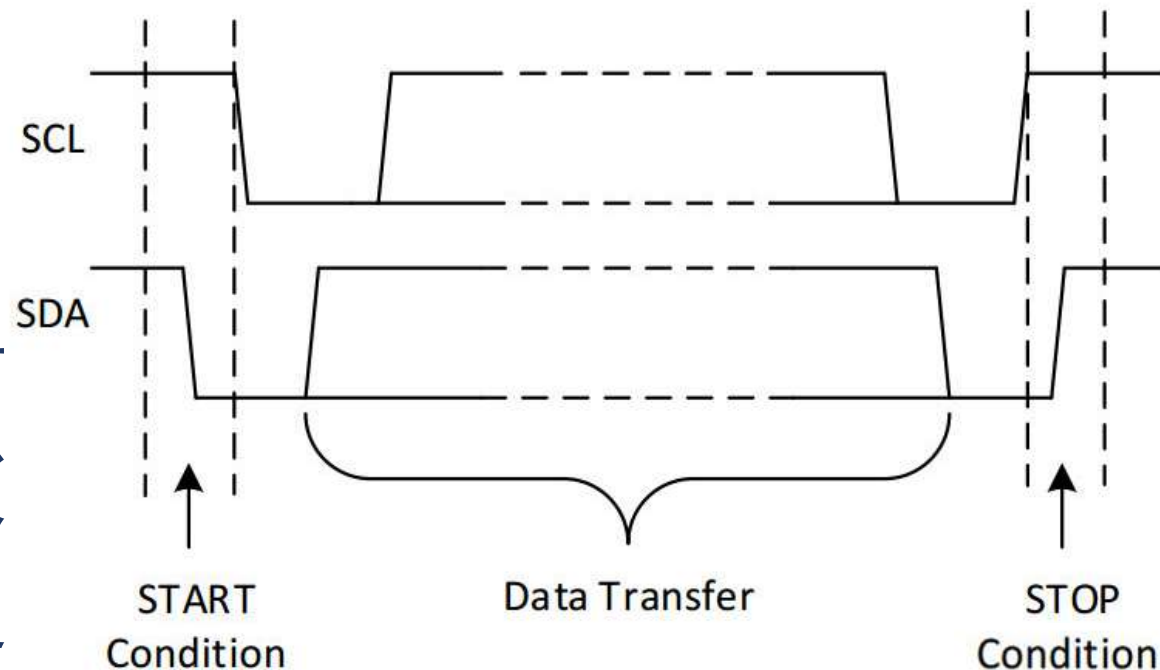


Arduino的I2C通信

■ 起始和停止条件

起始和停止条件都是由主机（master）发起产生。总线在起始条件之后处于忙碌状态，在停止条件之后又处于空闲状态

- 起始条件：SCL线是高电平时，SDA线从**高电平向低电平切换**
- 停止条件：SCL线是高电平时，SDA线从**低电平向高电平切换**
- 重复起始条件：重复起始条件发生在停止条件之前。主机想继续给从机发送消息时，一个字节传输完成后可以发送重复起始条件。

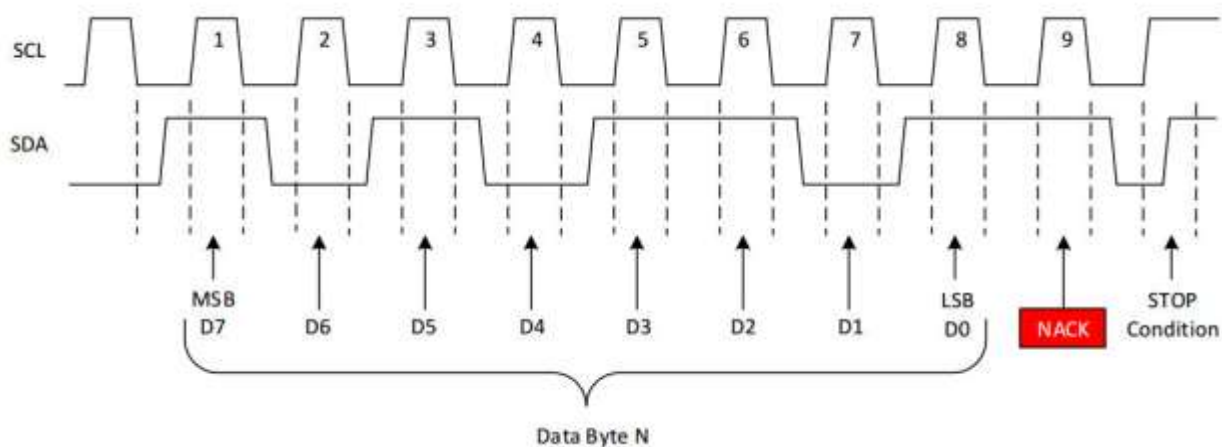
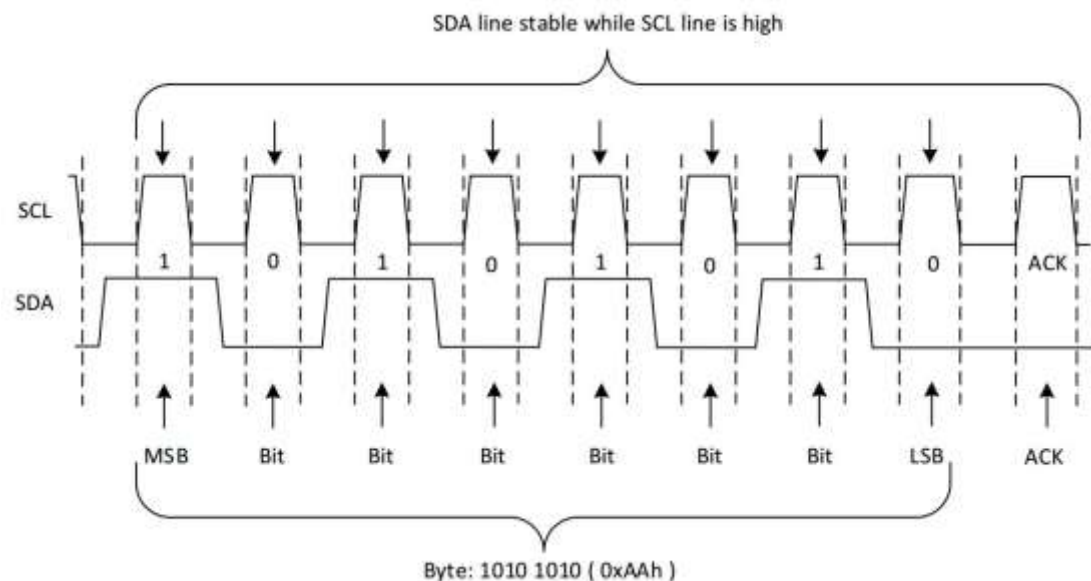




Arduino的I2C通信

■ 应答ACK和非应答NACK

- ACK: SCL第9位时钟高电平期间, SDA拉低
- NACK: SCL第9位时钟高电平期间, SDA仍然保持高电平。
 - 出现NACK时, 主机可以直接产生STOP条件终止以后的传输或者继续重新START开始一个新的传输

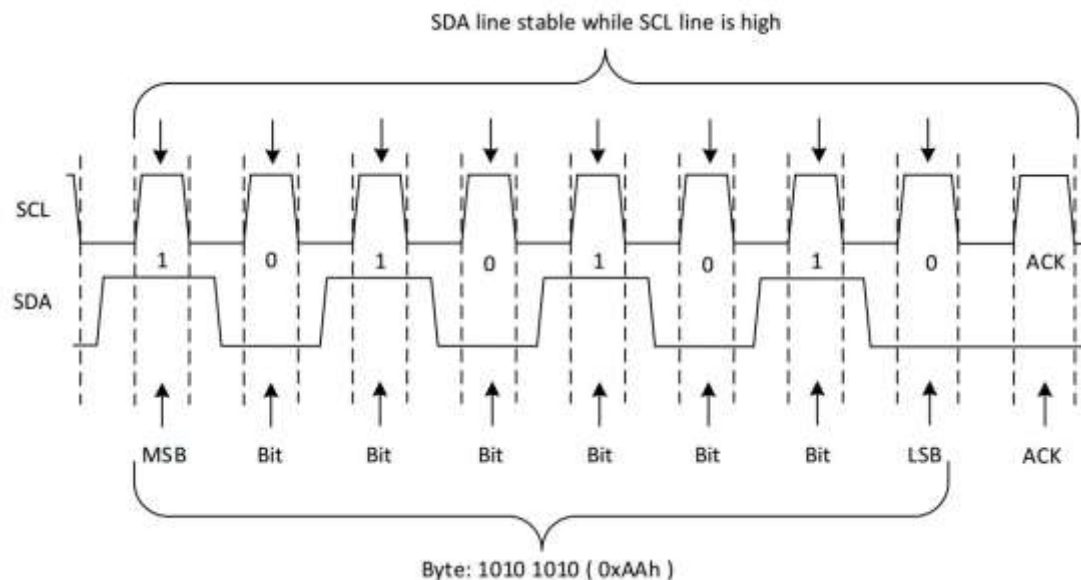
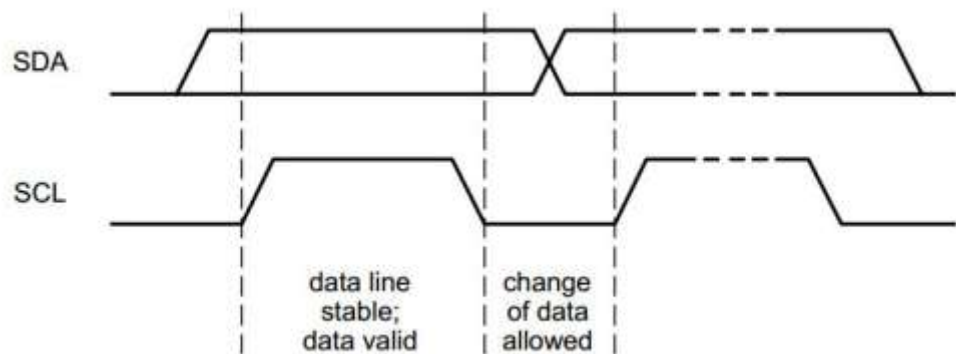




Arduino的I2C通信

■ 字节格式

- 每个字节8位，每个字节后必须跟一个响应位（ACK），首先传输的数据是最高位（MSB）
- SDA上的数据必须在SCL高电平周期时保持稳定，数据的高低电平翻转变化发生在SCL低电平时期。

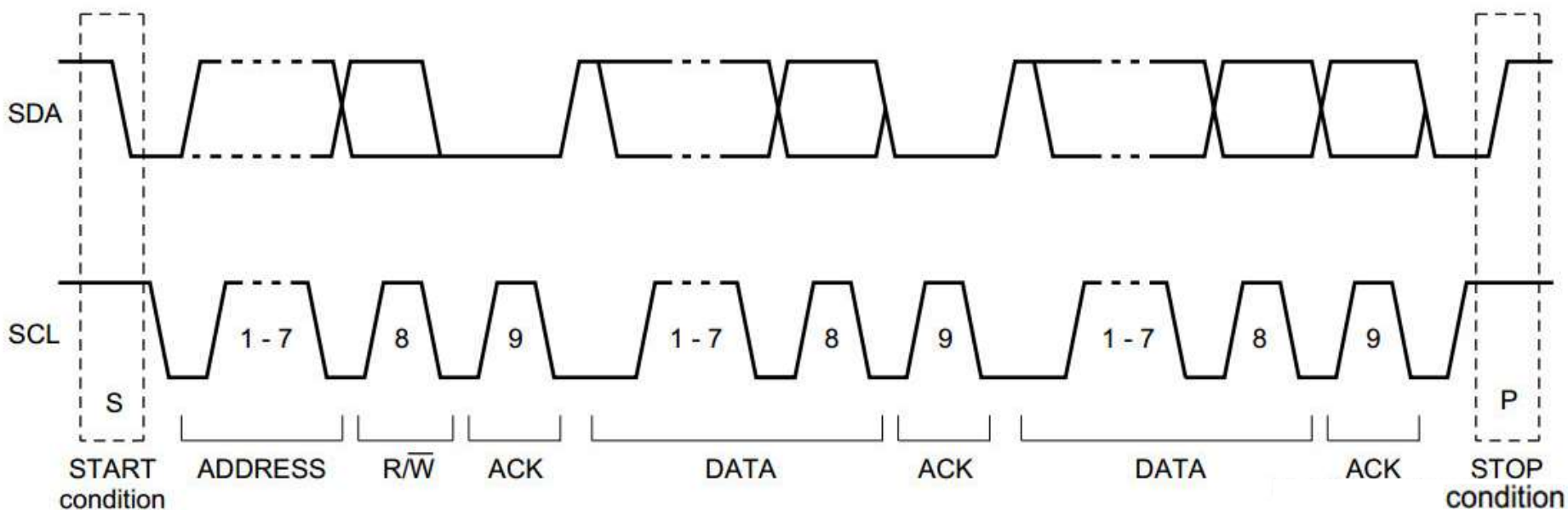




Arduino的I2C通信

■ 7-bit地址格式和读写位

- 最高位优先
- 地址位后有1-bit (R/W) 操作符，1表示读操作，0表示写操作
- (R/W) 操作符后有1-bit ACK/NACK





Arduino的I2C通信

■ 7-bit寻址数据传输

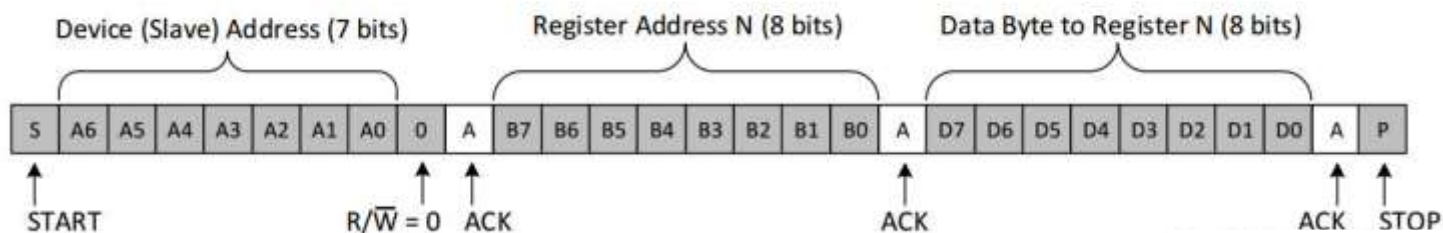
➤ 主机写数据-从机接收，传输方向不变

- 主机发送START条件+从机地址+R/W=0, 从机读取到该地址后回应ACK
- 主机将继续发送需要操作的寄存器地址，从机继续回应ACK
- 主机发送寄存器的数据每个byte从机都会回应ACK
- 发送完成后，主机发送STOP命令，将总线释放，完成写操作

☒ Master Controls SDA Line

☐ Slave Controls SDA Line

Write to One Register in a Device





Arduino的I2C通信

■ 7-bit寻址数据传输

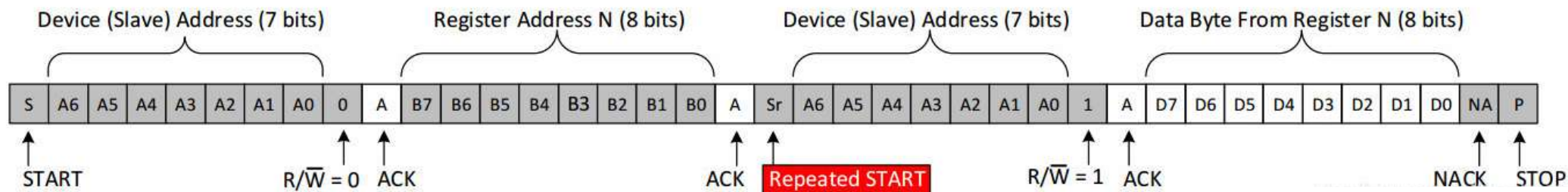
➤ 主机读数据-从机发送，传输方向改变

- 通过读操作获取从机地址及寄存器地址
- 主机发送重复开始+从机地址+R/W=1，从机返回ACK
- 主机释放SDA线转由从机控制，主机读取SDA总线进行数据接收
- 当主机接收完想要的的数据后，主机返回NACK，告诉从机释放SDA总线，随后主机发送STOP命令，将总线释放，完成读操作

■ Master Controls SDA Line

□ Slave Controls SDA Line

Read From One Register in a Device

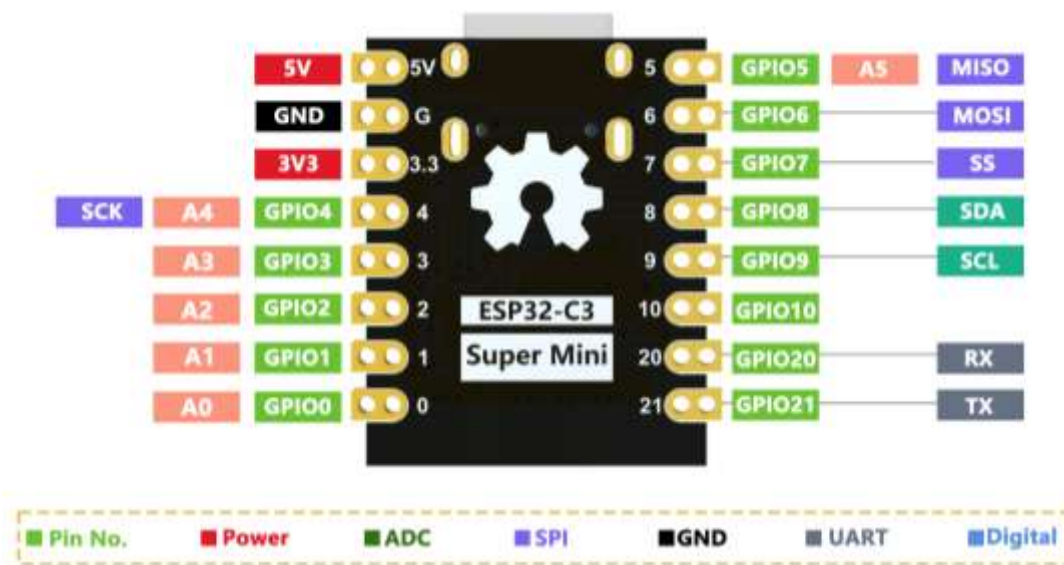




Arduino的I2C通信

■ ESP32的I2C接口

- ESP32C3 supermini共有两个I2C总线接口，可用作I2C主站或从站
- 这两路I2C的引脚可以任意设置，默认SDA（GPIO8）、SCL（GPIO9）
- SDA 和 SCL 线路处于低电平有效状态。





Arduino的I2C通信

■ ESP32的I2C接口

➤ ESP32 的 I2C 接口支持：

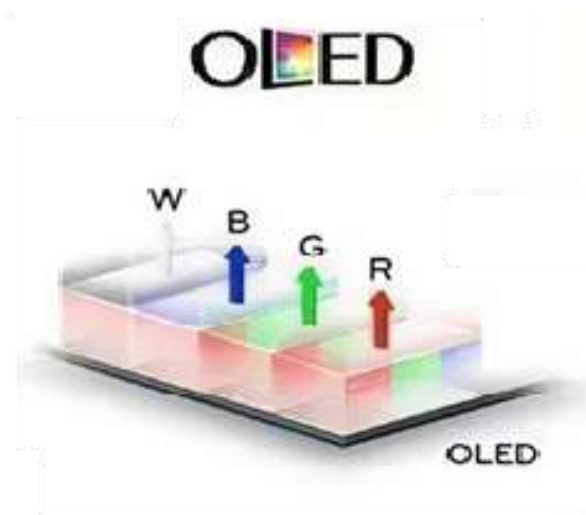
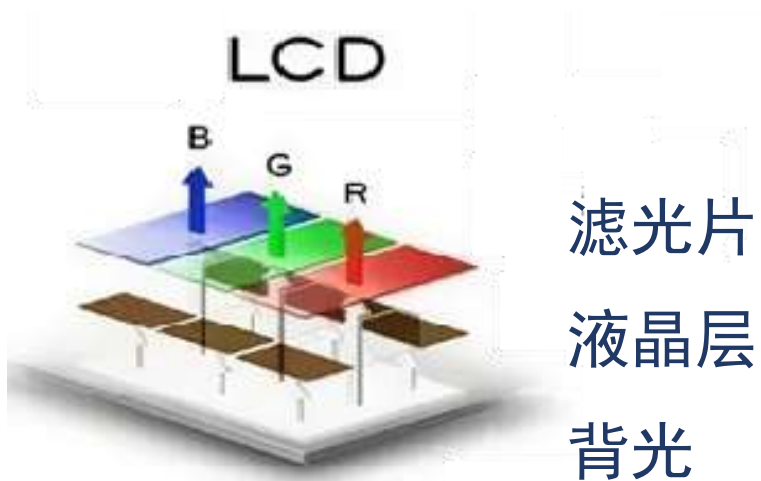
- 标准模式（100 Kbit/s）
- 快速模式（400 Kbit/s）
- 高达 5 MHz，但受 SDA 上拉强度限制
- 7 位/10 位寻址模式
- 双寻址模式。用户可以对命令寄存器进行编程以控制I2C接口，从而具有更大的灵活性.



OLED显示屏

■ OLED显示屏简介：

- 传统的LCD（液晶显示屏）利用液晶和不同颜色的滤光膜构成每个像素，滤光片和背光之间有个液晶层，背光白色。
- LED利用三种颜色的LED构成每个像素，相比于LCD，LED不需要背光源，可以做得更薄





OLED显示屏

■ SSD1306简介：

- SSD1306是一款OLED/PLED点阵显示屏的控制器，可以嵌入在屏幕中，用于执行接收数据、显示存储、扫描刷新等任务



VCC: 3.3V

GND: 接地

SCL: 串行时钟

SDA: 串行数据



OLED显示屏

■ SSD1306简介：

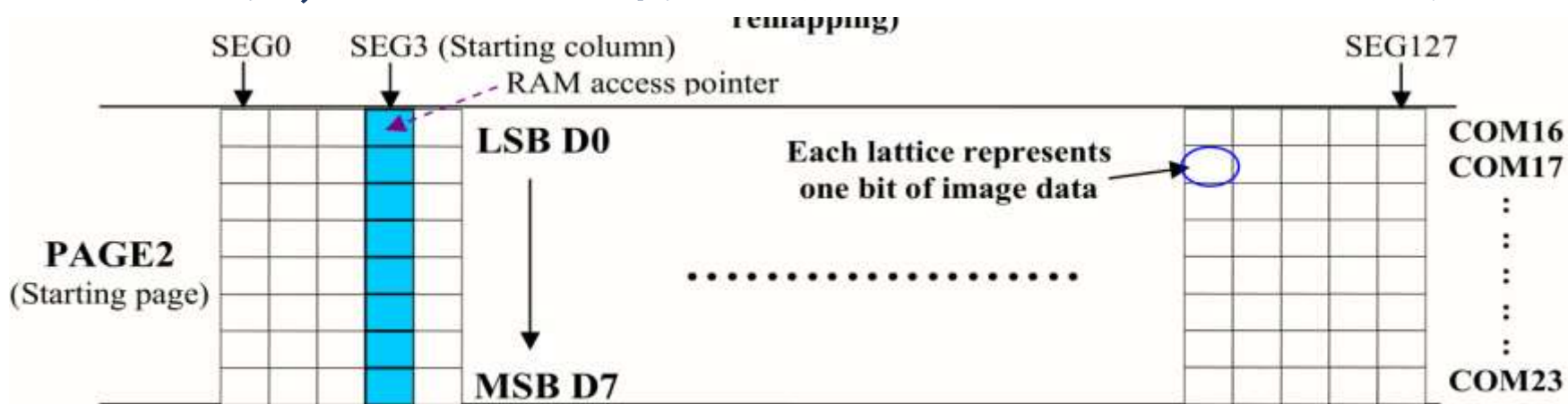
- 驱动接口：128个SEG引脚和64个COM引脚，对应128*64像素点阵显示屏
- 内置显示存储器（GDDRAM）： 12864 bit（1288 Byte）SRAM
- 供电：VDD=1.65~3.3V（提供给IC 逻辑电路）
- 通信接口：8位6800/8080并行接口（一次发送一个字节），3/4线SPI接口，I2C接口（一次发送一个位）



OLED显示屏

■ 显示原理：

- 水平方向分布了128列像素点，垂直方向分布了64行像素点。
- 驱动芯片以一次8个像素点的方式点亮屏幕，也就是每次画垂直方向的8个像素点，所以我们在画点的时候Y的取值为0-7，X的取值为0-127。
- 为方便起见，设计者将同一列的8个点阵编成一组，用一个8bit数表示，这样的一行8bit数（即128*8bits）被称为**一页**。

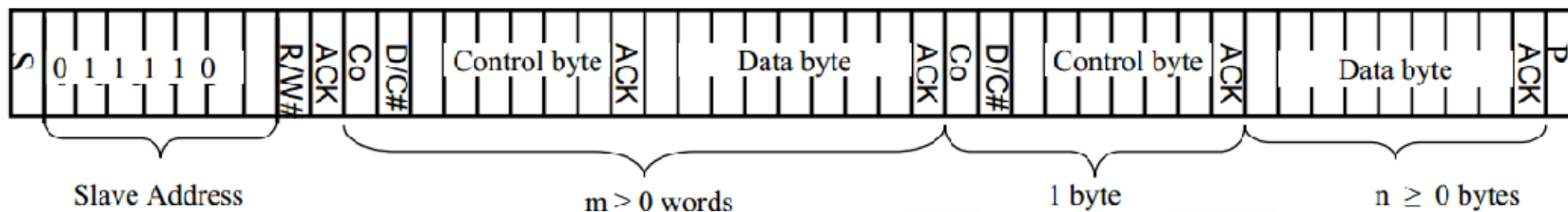
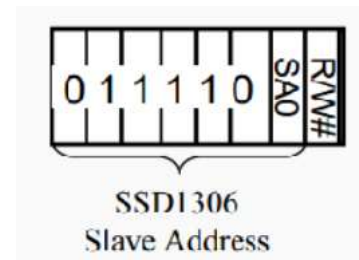




OLED显示屏

■ SSD1306的I2C时序：

- 起始条件 (S)
- 主机发送7位从机地址+1位读写位
- 接收应答ACK，判断从机是否收到
- 发送一个控制字节，确定后面的有效字节是命令还是数据，之后ACK
- 发送有效字节，每发一个字节进行一次ACK
- 结束条件 (P)



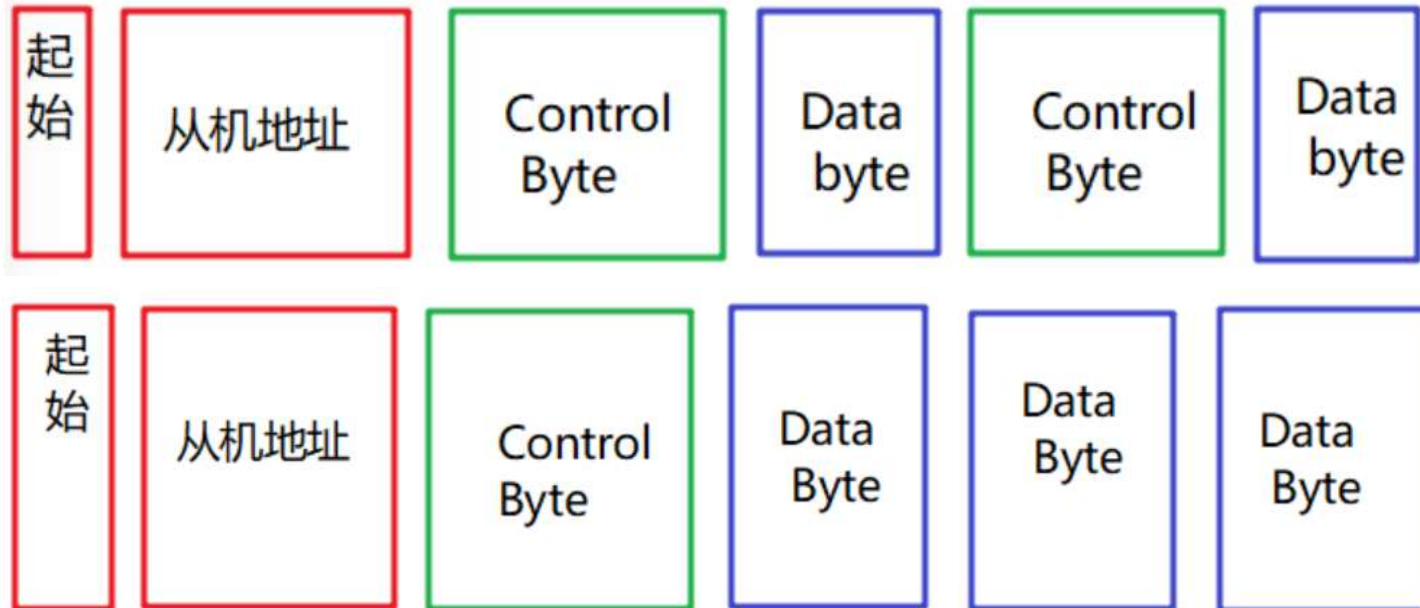
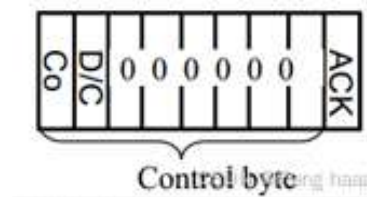


OLED显示屏

■ SSD1306的I2C时序：

➤ 控制字节中只有前两位Co、D/C有效

- Co=1时，后续有效字节与控制字节交替发送；Co=0时，后续连续发送有效字节
- D/C指定后面的有效字节是命令还是数据

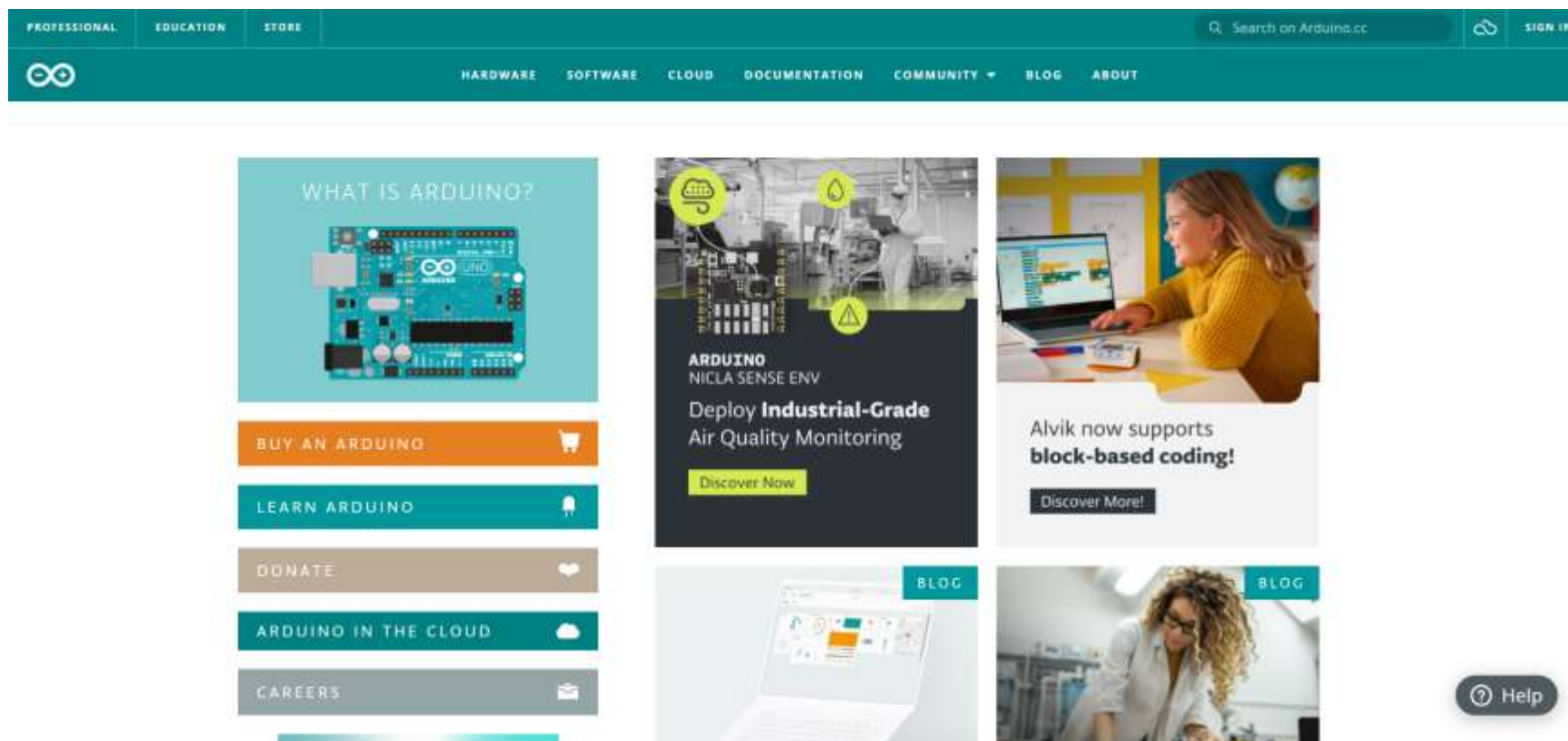




Arduino的库文件

■ Arduino开源网站：

- Arduino官网提供大量开源库文件，可以在网站上查询、下载后从Arduino IDE中导入

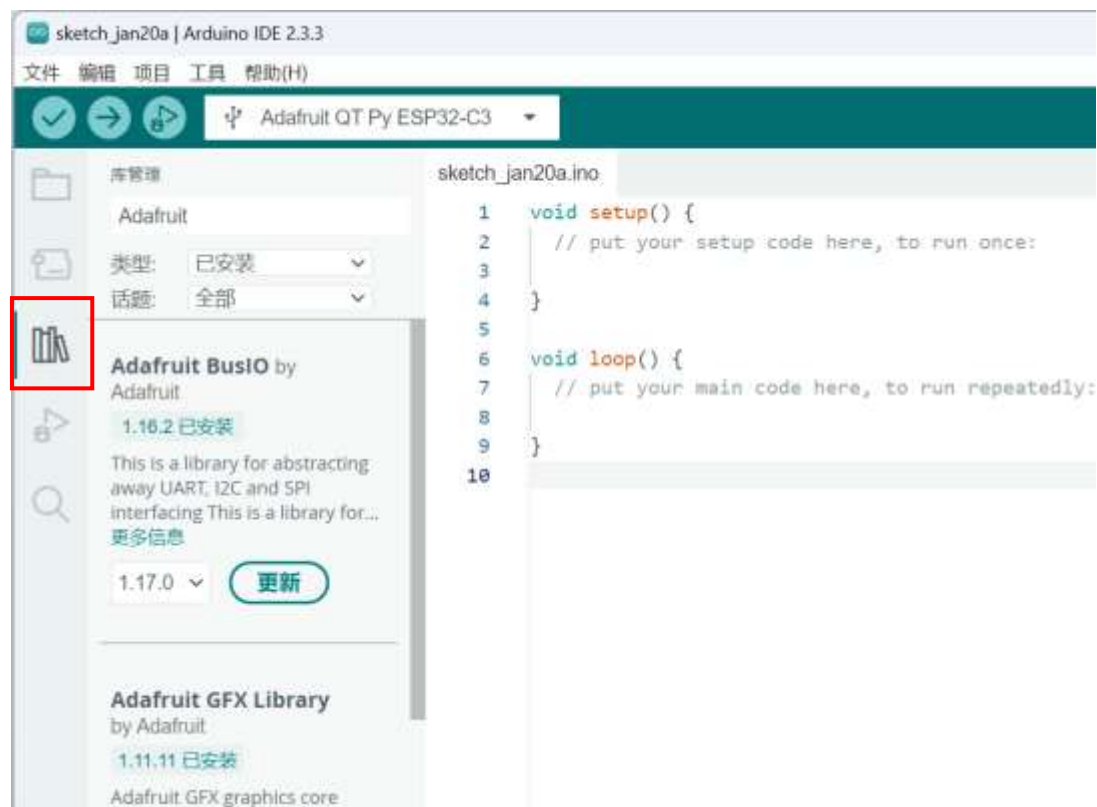




Arduino的库文件

■ Arduino IDE自带库管理：

- Arduino 2.x版本新加入库管理栏，可以直接在开发界面在线下载安装需要的库文件





Arduino的库文件

■ 安装OLED支持库：

➤ 方式1：从库管理器中下载SSD1306需要的两项依赖库：

- Adafruit SSD1306（版本2.5.13）
- Adafruit GFX Library（版本1.11.11）

Adafruit SSD1306 by
Adafruit

2.5.13 已安装

SSD1306 oled driver library for
monochrome 128x64 and
128x32 displays SSD1306 oled...
[更多信息](#)

2.5.13 ▼

移除

Adafruit GFX Library
by Adafruit

1.11.11 已安装

Adafruit GFX graphics core
library, this is the 'core' class
that all our other graphics...
[更多信息](#)

1.11.11 ▼

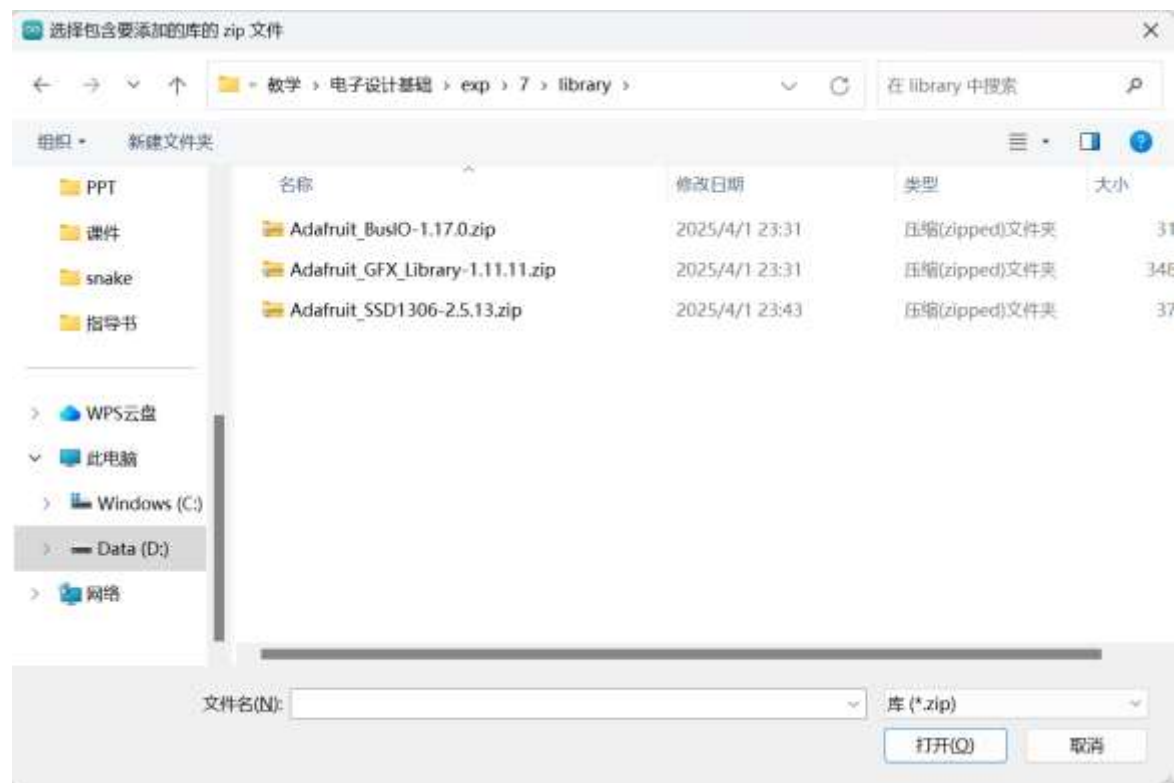
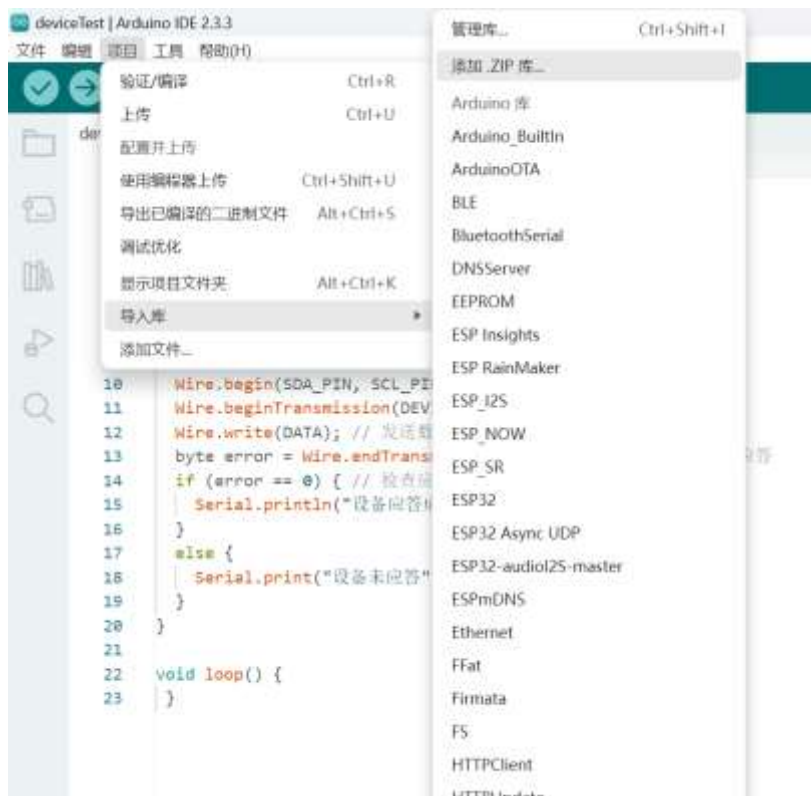
移除



Arduino的库文件

■ 安装OLED支持库：

- 方式2：也可以安装已下载好的库文件ZIP包进行安装
- 选择项目->导入库->添加.ZIP库，将library中库文件全部安装





Arduino的I2C通信

■ Arduino Wire库

➤ Arduino使用Wire库实现I2C通信

- Wire.begin() : 初始化I2C库，设置为主设备或从设备。
- Wire.requestFrom(address, quantity): 请求从指定地址读取指定数量的数据。
- Wire.beginTransmission(address): 开始与指定地址的设备通信。
- Wire.write(data) : 向设备发送数据。
- Wire.endTransmission() : 结束数据传输。
- Wire.available() : 检查接收缓冲区中可读取的数据字节数。
- Wire.read(): 读取接收到的数据。



Arduino的库文件

■ Adafruit常用函数：

函数名	效果
clearDisplay	清除显示，该方法仅清除 Arduino 缓存，不会立即显示在屏幕上，可以通过调用 display 来立即清除
display	显示内容，这个方法才是真正把绘制内容画在 OLED 屏幕上（非常重要）
setTextSize	设置字体大小
setFont	设置字体
setTextColor	设置字体颜色
drawPixel	绘制像素点
drawChar	绘制单个字符



Arduino的库文件

■ Adafruit常用函数：

函数名	效果
drawFastHLine	绘制水平线
drawFastVLine	绘制垂直线
setCursor	设置光标位置
getTextBounds	计算字符串在当前字体大小下的像素大小，返回左上角坐标以及宽度高度像素值
startscrollright	滚动到右边
startscrollleft	滚动到左边
stopscroll	停止滚动



Arduino的I2C通信

■ 实验1：I2C通信实验

- OLED显示模块可作为从机与作为主机的Arduino实验板通讯

■ 电路连接：

- 显示模块按照图示方向插入底板
- DISPLAY实验板的3.3V、GND、SCL、SDA分别接ESP32开发板的3.3V_OUT、GND、9SCL、8SDA端口。
- SCL和SDA引脚与示波器的两个通道相连





Arduino的I2C通信

■ I2C通信实验

- OLED屏幕的默认地址为**0x3c**，当向此地址读写数据时，OLED会进行应答；当向其它地址读写数据时，没有设备会进行应答
- 使用示波器观察SCL**第九个高电平期间**时SDA的电平
- 修改实验代码中的设备地址，重新观察示波器现象

```
1  #include <Wire.h>
2
3  #define SDA_PIN 8
4  #define SCL_PIN 9
5  #define DEVICE_ADDR 0x3C
6  #define DATA 0x33
7  void setup() {
8      // put your setup code here, to run once:
9      Serial.begin(9600);
10     Wire.begin(SDA_PIN, SCL_PIN);
11
12 }
13
14 void loop() {
15     // put your main code here, to run repeatedly:
16     delay(1000);
17     Wire.beginTransmission(DEVICE_ADDR);
18     Wire.write(DATA);
19     byte error = Wire.endTransmission();
20     if (error == 0) {
21         Serial.println("设备应答成功");
22     } else {
23         Serial.print("设备未应答");
24     }
25
26 }
```



Arduino的I2C通信

■ 实验结果

➤ 当存在设备应答时

Wire.endTransmission()返回0
示波器观察到有ACK应答

➤ 地址0x3C测试

- 第九个高电平期间时SDA的电平为低
- 有ACK应答

➤ 地址0x3D测试

- 第九个高电平期间时SDA的电平为高
- 无ACK应答





字符与图像显示

■ 实验2：点亮像素点：

➤ 实验参考代码：

```
2  #include <Wire.h> // Wire库写I2C驱动
3  #include <Adafruit_GFX.h>
4  #include <Adafruit_SSD1306.h>
5  #define SCREEN_WIDTH 128 // OLED 显示屏宽度
6  #define SCREEN_HEIGHT 64 // OLED 显示屏高度
7
8  // I2C
9  #define OLED_RESET -1
10 Adafruit_SSD1306 oled(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
11
12 void setup()
13 {
14     oled.begin();
15     oled.clearDisplay();
16     oled.drawPixel(10,10,SSD1306_WHITE);
17     oled.display();
18 }
19
20 void loop()
21 {
22 }
23 |
```




字符与图像显示

■ 点亮像素点：

➤ 实验结果：





字符与图像显示

■ 实验2：绘制水平线：

➤ 实验代码：

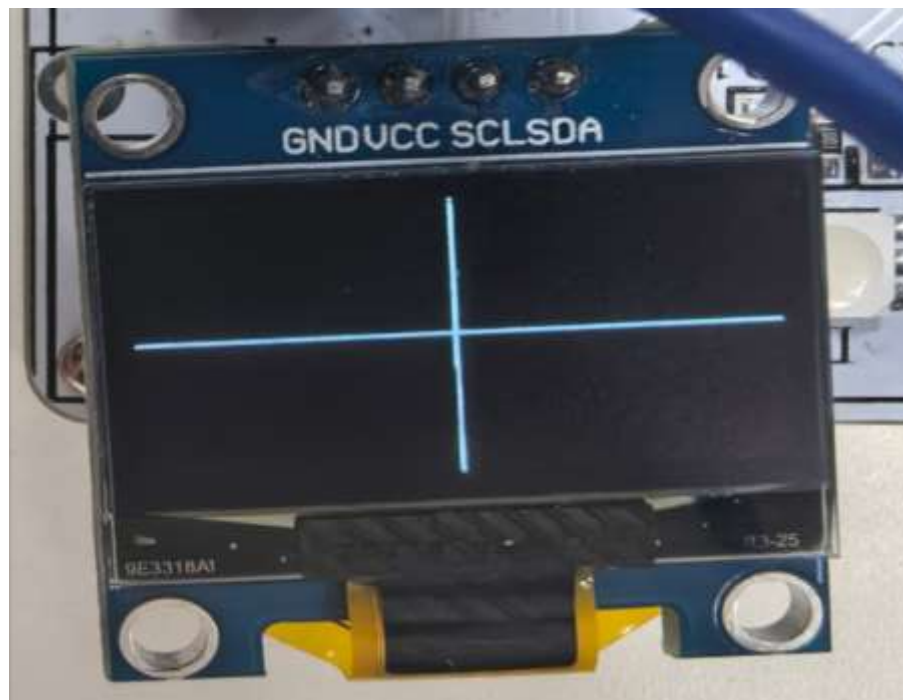
```
1  #include <Arduino.h>
2
3  #include <SPI.h>
4  #include <Wire.h>
5  #include <Adafruit_GFX.h>
6  #include <Adafruit_SSD1306.h>
7  #define SCREEN_WIDTH 128 // OLED 显示屏宽度
8  #define SCREEN_HEIGHT 64 // OLED 显示屏高度
9
10 // 软件SPI总线
11 #define OLED_RESET -1
12 Adafruit_SSD1306 oled(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
13
14 void setup()
15 {
16     oled.begin();
17     oled.clearDisplay();
18     oled.drawFastHLine(0, 0, 128, SSD1306_INVERSE);
19     oled.drawFastHLine(0, 15, 128, SSD1306_INVERSE);
20     oled.drawFastHLine(0, 16, 128, SSD1306_INVERSE);
21     oled.drawFastHLine(0, 63, 128, SSD1306_INVERSE);
22     oled.display();
23 }
24
25 void loop()
26 {
27 }
```



字符与图像显示

■ 绘制水平线：

➤ 实验结果：





字符与图像显示

■ 实验2：显示字符串：

➤ 实验代码：

```
1  #include <Arduino.h>
2
3  #include <SPI.h>
4  #include <Wire.h>
5  #include <Adafruit_GFX.h>
6  #include <Adafruit_SSD1306.h>
7  #define SCREEN_WIDTH 128 // OLED 显示屏宽度
8  #define SCREEN_HEIGHT 64 // OLED 显示屏高度
9
10 // I2C
11 #define OLED_RESET -1
12 Adafruit_SSD1306 oled(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
13
14 void setup()
15 {
16     oled.begin();
17     oled.clearDisplay();
18
19     oled.setCursor(5, 50);
20     oled.setTextSize(1.5);
21     oled.setTextColor(WHITE);
22     oled.println("HelloWorld");
23
24     oled.display();
25 }
26
27 void loop()
28 {
29 }
```



字符与图像显示

■ 显示字符串：

➤ 实验结果：





实验3：控制点坐标移动

■ 使用ESP32开发板控制点坐标移动：

➤ 要求实现以下功能：

- 使用开发板上的S1、S2按键进行操作；
- 在初始化中在屏幕中心显示一个点；
- 在屏幕上方合适位置显示作者的学号；
- S1、S2键分别控制显示点坐标的左右移动；

请同学自行编写程序代码，AI辅助检查





实验3：控制点坐标移动

■ 电路连接：

- 显示模块按照图示方向插入底板
- DISPLAY实验板的3.3V、GND、SCL、SDA分别接ESP32开发板的3.3V、GND、9SCL、8SDA端口
- DISPLAY实验板的按钮BTN1、BTN2分别连接ESP32的5、6端口



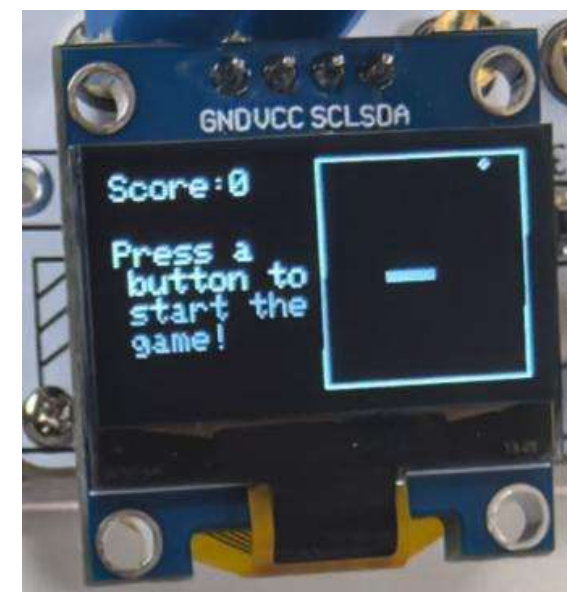


实验4：Arduino贪吃蛇游戏

■ 使用ESP32开发板制作贪吃蛇小游戏：

➤ 要求实现以下功能：

- 使用开发板上的S1、S2按键进行操作；
- 按下任意键开始游戏；
- 游戏开始时蛇的体长较短，随着游戏的进行，体长会不断增加；
- 游戏途中屏幕上随机生成果实，蛇的头部吃掉果实后，蛇的体长增加1，场上立即生成新的果实；
- S1、S2键分别控制蛇头的左、右转向；
- 当蛇头穿越自身体节或超过边界时，游戏结束，按任意键重新开始游戏。





实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
2  #include <Wire.h>
3  #include <Adafruit_GFX.h>
4  #include <Adafruit_SSD1306.h>
5
6  #define SCREEN_WIDTH      128 // OLED display width, in pixels
7  #define SCREEN_HEIGHT    64  // OLED display height, in pixels
8
9  // Declaration for an SSD1306 display connected to I2C (SDA, SCL pins)
10 #define OLED_RESET        -1 // Reset pin # (or -1 if sharing Arduino reset pin)
11 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
12
13 const byte buttonPins[] = {5, 6}; // RIGHT, LEFT
14
15 typedef enum {
16     START,
17     RUNNING,
18     GAMEOVER
19 } State;
20
21 typedef enum {
22     LEFT,
23     UP,
24     RIGHT,
25     DOWN
26 } Direction;
27
28 #define SNAKE_PIECE_SIZE      3
29 #define MAX_SANKE_LENGTH    165
30 #define MAP_SIZE_X          20
31 #define MAP_SIZE_Y          20
32 #define STARTING_SNAKE_SIZE  5
33 #define SNAKE_MOVE_DELAY    30
34
35 State gameState;
36
37 int8_t snake[MAX_SANKE_LENGTH][2];
38 uint8_t snake_length;
39 Direction dir;
40 Direction newDir;
41
42 int8_t fruit[2];
43
```

变量声明与库函数调用



实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
44 void setup() {  
45     Serial.begin(9600);  
46     if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3D)) {  
47         Serial.println(F("SSD1306 allocation failed"));  
48         for(;;);  
49     }  
50  
51     for (byte i = 0; i < 2; i++) {  
52         pinMode(buttonPins[i], INPUT_PULLUP);  
53     }  
54  
55     randomSeed(analogRead(A0));  
56  
57     setupGame();  
58 }
```

程序初始化，打开串口，
点亮OLED屏，
生成随机数种子



实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
60 void setupGame() {  
61     gameState = START;  
62     dir = RIGHT;  
63     newDir = RIGHT;  
64     resetSnake();  
65     generateFruit();  
66     display.clearDisplay();  
67     drawMap();  
68     drawScore();  
69     drawPressToStart();  
70     display.display();  
71 }
```

调用函数，
游戏界面初始化

```
73 void resetSnake() {  
74     snake_length = STARTING_SNAKE_SIZE;  
75     for(int i = 0; i < snake_length; i++) {  
76         snake[i][0] = MAP_SIZE_X / 2 - i;  
77         snake[i][1] = MAP_SIZE_Y / 2;  
78     }  
79 }
```

在屏幕中心生成初始蛇



实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
81  int moveTime = 0;
82  void loop() {
83      switch(gameState) {
84          case START:
85              if(buttonPress()) gameState = RUNNING;
86              break;
87
88
89          case RUNNING:
90              moveTime++;
91              readDirection();
92              if(moveTime >= SNAKE_MOVE_DELAY) {
93                  dir = newDir;
94                  display.clearDisplay();
95                  if(moveSnake()) {
96                      gameState = GAMEOVER;
97                      drawGameOver();
98                      delay(1000);
99                  }
100                 drawMap();
101                 drawScore();
102                 display.display();
103                 checkFruit();
104                 moveTime = 0;
105             }
106             break;
```

游戏分为三种状态：未开始→进行中→游戏结束，重复循环

```
108         case GAMEOVER:
109             if(buttonPress()) {
110                 delay(500);
111                 setupGame();
112                 gameState = START;
113             }
114             break;
115     }
116
117     delay(10);
118 }
```



实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
120 bool buttonPress() {
121     static int done[2] = {0};
122     for (byte i = 0; i < 2; i++) {
123         byte buttonPin = buttonPins[i];
124         if (digitalRead(buttonPin) == LOW)
125         {
126             if (!done[i]) {
127                 done[i] = 1;
128                 return true;
129             }
130         } else if (done[i]) {
131             done[i] = 0;
132         }
133     }
134     return false;
135 }
136 void readDirection() {
137     static int done[2] = {0};
138     for (byte i = 0; i < 2; i++) {
139         byte buttonPin = buttonPins[i];
140         if (digitalRead(buttonPin) == LOW )
141         {
142             if (!done[i]) {
143                 newDir = (Direction)((((int)dir+i*2+1) % 4);
144             }
145             done[i] = 1;
146             return;
147         } else {
148             done[i] = 0;
149         }
150     }
151 }
```

判断是否有按键按下，任一按键被按下后返回值true

游戏进行中，根据被按下的按键完成蛇头的顺/逆时针90度转向



实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
140 bool moveSnake() {
141     int8_t x = snake[0][0];
142     int8_t y = snake[0][1];
143
144     switch(dir) {
145         case LEFT:
146             x -= 1;
147             break;
148         case UP:
149             y -= 1;
150             break;
151         case RIGHT:
152             x += 1;
153             break;
154         case DOWN:
155             y += 1;
156             break;
157     }
158
159     if(collisionCheck(x, y))
160         return true;
```

控制蛇头往所朝的方向前进，前进后
判断头部是否发生碰撞

```
162     for(int i = snake_length - 1; i > 0; i--) {
163         snake[i][0] = snake[i - 1][0];
164         snake[i][1] = snake[i - 1][1];
165     }
166
167     snake[0][0] = x;
168     snake[0][1] = y;
169     return false;
170 }
```

蛇的身体向前一格



实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
181 void generateFruit() {
182     bool b = false;
183     do {
184         b = false;
185         fruit[0] = random(0, MAP_SIZE_X);
186         fruit[1] = random(0, MAP_SIZE_Y);
187         for(int i = 0; i < snake_length; i++) {
188             if(fruit[0] == snake[i][0] && fruit[1] == snake[i][1]) {
189                 b = true;
190                 continue;
191             }
192         }
193     } while(b);
194 }
195
196 bool collisionCheck(int8_t x, int8_t y) {
197     for(int i = 1; i < snake_length; i++) {
198         if(x == snake[i][0] && y == snake[i][1]) return true;
199     }
200     if(x < 0 || y < 0 || x >= MAP_SIZE_X || y >= MAP_SIZE_Y) return true;
201     return false;
202 }
```

在随机位置生成果实，若该位置已有蛇体占据则重新生成

蛇头经过身体、超过屏幕范围都视为发生碰撞



实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
204 void drawMap() {
205     int offsetMapX = SCREEN_WIDTH - SNAKE_PIECE_SIZE * MAP_SIZE_X - 2;
206     int offsetMapY = 2;
207
208     display.drawRect(fruit[0] * SNAKE_PIECE_SIZE + offsetMapX,
209                     fruit[1] * SNAKE_PIECE_SIZE + offsetMapY,
210                     SNAKE_PIECE_SIZE, SNAKE_PIECE_SIZE, SSD1306_INVERSE);
211     display.drawRect(offsetMapX - 2, 0, SNAKE_PIECE_SIZE * MAP_SIZE_X + 4,
212                     SNAKE_PIECE_SIZE * MAP_SIZE_Y + 4, SSD1306_WHITE);
213     for(int i = 0; i < snake_length; i++) {
214         display.fillRect(snake[i][0] * SNAKE_PIECE_SIZE + offsetMapX,
215                         snake[i][1] * SNAKE_PIECE_SIZE + offsetMapY,
216                         SNAKE_PIECE_SIZE, SNAKE_PIECE_SIZE, SSD1306_WHITE);
217     }
218 }
```

绘制屏幕上的蛇、果实和边界



实验4：Arduino贪吃蛇游戏

■ 实验代码：

```
220 void drawScore() {  
221     display.setTextSize(1);  
222     display.setTextColor(SSD1306_WHITE);  
223     display.setCursor(2,2);  
224     display.print(F("Score:"));  
225     display.println(snake_length - STARTING_SNAKE_SIZE);  
226 }
```

绘制得分表

```
228 void drawPressToStart() {  
229     display.setTextSize(1);  
230     display.setTextColor(SSD1306_WHITE);  
231     display.setCursor(2,20);  
232     display.print(F("Press a\n button to\n start the\n game!"));  
233 }
```

绘制开始界面

```
235 void drawGameOver() {  
236     display.setTextSize(1);  
237     display.setTextColor(SSD1306_WHITE);  
238     display.setCursor(2,50);  
239     display.println(F("GAMEOVER"));  
240 }
```

绘制结束界面



实验4：Arduino贪吃蛇游戏

■ 添加灯光效果功能

- 每吃到一次果实，三色LED灯绿灯亮起1s后熄灭
- 每次碰撞失败时，三色LED灯红灯亮起直至重新开始游戏后熄灭

■ 添加45度转弯功能

- 按下一次右转或左转按键，蛇头顺时针或逆时针45度转向

■ 添加穿越边界功能（选做）

- 蛇头到达边界时不发生碰撞，而是从另一边对应位置处出来



实验5（选做）：双人贪吃蛇游戏

■ 实现双人游戏功能

- 两人各自使用一块显示控制板，游戏显示屏中同时出现两条贪吃蛇、一个果实，和双方得分
- 各自的按键控制各自一条蛇的转向
- 两块OLED屏中显示同样的图像，同步变化
- 蛇头碰到边界或者另一条蛇的身体扣一分，最多扣4分
- 硬件方案
 - 选项1：一个Arduino主控板，Arduino连接两块显示控制板
 - 选项2：两个Arudino主控板，主控板间通过串口1交换数据
- 软件代码可使用生成式大模型AI进行辅助编程